

AfterReload — Documentación de Fin de Curso

Ciclo Formativo: Desarrollo de Aplicaciones Web (DAW)

Alumno: Alejandro

Proyecto: AfterReload — Plataforma web de matchmaking para Counter-Strike

Tecnología principal: Laravel 12 + PHP 8.2

Fecha: Mayo 2026

Índice

1. [Introducción](#)
 2. [Planificación](#)
 3. [Análisis](#)
 4. [Diseño](#)
 5. [Implementación](#)
 6. [Validación](#)
 7. [Pruebas](#)
 8. [Resultados](#)
-

1. Introducción

1.1 Descripción del proyecto

AfterReload es una plataforma web de matchmaking y gestión de servidores diseñada para comunidades de Counter-Strike. Su propósito es centralizar en un único sistema la autenticación de jugadores mediante Steam, la organización de partidas a través de lobbies, el control remoto de servidores de juego y el registro de estadísticas y progresión de los jugadores.

El nombre hace referencia a la acción de recargar el arma justo después de eliminar al último enemigo, un momento icónico del juego que simboliza la preparación para la siguiente ronda.

1.2 Contexto y motivación

Las comunidades de Counter-Strike autogestionadas dependen habitualmente de herramientas externas y dispersas para organizar partidas: grupos de Discord para coordinar jugadores, paneles de administración para gestionar servidores, hojas de cálculo para estadísticas y bots para gestión de acceso. Esta dispersión genera fricciones, errores de coordinación y una experiencia de usuario degradada.

AfterReload surge como respuesta a esta problemática. El objetivo es ofrecer una plataforma propia que integre todos estos procesos en un entorno web cohesionado, accesible desde el navegador y sin dependencias de herramientas de terceros para las operaciones cotidianas.

1.3 Alcance del proyecto

El sistema cubre los siguientes ámbitos funcionales:

- Autenticación de usuarios mediante la identidad de Steam (OAuth 2.0).
- Gestión de servidores de juego diferenciando entre servidores públicos y matchmaking.

- Sistema de lobbies con selección de equipo, indicador de estado de preparación y bloqueo automático durante partidas en curso.
- Lobbies públicos de capacidad configurable con join/leave libre y validación de presencia para la asignación de puntos.
- Integración con el plugin Get5 para la orquestación de partidas competitivas.
- Control remoto de servidores mediante el protocolo RCON.
- Procesamiento automático de resultados en servidores públicos a partir de logs del motor de juego, con filtro de presencia en lobby.
- Procesamiento manual de logs mediante comando Artisan.
- Sistema de puntos y créditos como mecanismo de progresión del jugador.
- Panel de administración centralizado para la gestión de usuarios, roles, baneos y servidores.
- Formulario de contacto con envío de email al administrador mediante SMTP.
- Eliminación voluntaria de cuenta por parte del usuario.
- Interfaz multiidioma con soporte para español e inglés.

1.4 Tecnologías empleadas

Capa	Tecnología	Versión
Backend	Laravel (PHP)	12.0 / PHP 8.2+
ORM	Eloquent	Laravel 12
Frontend	Blade + Tailwind CSS	Tailwind 4.0
Build	Vite	7.0.7
HTTP client	Axios	1.11.0
Base de datos	SQLite / MySQL	-
Autenticación	Steam OAuth (Socialite)	^4.3
Testing	PHPUnit	11.5.3
Servidor web	Nginx + PHP-FPM	8.2
Control de versiones	Git	-

2. Planificación

2.1 Metodología de trabajo

El proyecto se desarrolló de forma iterativa e incremental, sin aplicar una metodología formal de gestión de proyectos como Scrum o Kanban, dado el carácter individual del desarrollo. Sin embargo, sí se siguió una progresión estructurada en fases con hitos claros, priorizando la funcionalidad sobre la completitud.

Cada fase se inició una vez que la anterior producía un resultado estable y verificable. Esto permitió detectar problemas de integración de forma temprana y ajustar el alcance sin comprometer la integridad del sistema.

2.2 Fases del proyecto

Fase 1 — Base del sistema (semanas 1-2)

Objetivos:

- Creación del proyecto Laravel desde cero.
- Definición del esquema inicial de base de datos con sus migraciones.
- Configuración de autenticación mediante Steam (Socialite).
- Estructura de rutas, controladores y vistas base con Blade.
- Layout principal con sidebar, topbar y sistema de navegación.

Resultado: Sistema funcional con login via Steam, gestión de sesiones y estructura MVC completa.

Fase 2 — Gestión de servidores y panel admin (semanas 3-4)

Objetivos:

- Modelo `Server` con campos para IP, puerto, tipo, estado y capacidad.
- Panel de administración para crear, editar y eliminar servidores.
- Vista de listado de servidores con actualización automática.
- Sistema de roles de usuario (user, admin, betatester, store).
- Middleware de control de acceso y gestión de usuarios baneados.

Resultado: Panel de administración operativo con CRUD de servidores y control de usuarios.

Fase 3 — Lobbies y matchmaking (semanas 5-6)

Objetivos:

- Modelo `Lobby` y tabla pivote `lobby_user` para la relación N:M entre usuarios y lobbies.
- Controlador de lobby con lógica de entrada, salida, selección de equipo y estado de preparación.
- Bloqueo de lobby durante partida activa y lobbies públicos de capacidad libre.
- Integración con Get5 para construcción de configuraciones de partida.
- Webhook de eventos Get5 para procesamiento de resultados competitivos.
- Sincronización de puntos con base de datos de plugin SourceMod (opcional).

Resultado: Flujo completo de matchmaking desde el lobby hasta la conclusión del match y aplicación de puntos.

Fase 4 — Servidores públicos y RCON (semanas 7-8)

Objetivos:

- Implementación del cliente RCON para comunicación con servidores de juego.
- Servicio de procesamiento de logs de servidores públicos con filtro de presencia.
- Detección automática de finales de partida y aplicación de recompensas.
- Comando Artisan para procesamiento manual de logs.
- Servicio de verificación del estado de runtime del servidor local (LGSM).

Resultado: Los servidores públicos pueden detectar y registrar partidas finalizadas sin intervención manual.

Fase 5 — Contacto, perfil y refinamientos (semanas 9-10)

Objetivos:

- Formulario de contacto con envío de email vía SMTP y validación AJAX.
- Eliminación de cuenta con modal de confirmación.

- Traducción completa de la interfaz al español e inglés.
- Pruebas automatizadas con PHPUnit para el procesador de logs y el controlador de lobby.
- Revisión general del sistema, limpieza de código y ajustes finales.

Resultado: Sistema completo, documentado y con cobertura de pruebas en los módulos críticos.

2.3 Diagrama de fases

Semana:	1	2	3	4	5	6	7	8	9	10
Fase 1:	[=====]									
Fase 2:		[=====]								
Fase 3:			[=====]							
Fase 4:				[=====]						
Fase 5:							[=====]			

2.4 Recursos empleados

- **Hardware:** Servidor Linux con Ubuntu 24.04 LTS para el despliegue.
- **Software de desarrollo:** Editor de código, terminal, navegador para pruebas.
- **Servicios externos:** Cuenta de desarrollador de Steam para OAuth, servidor CS:GO con Get5 instalado.
- **Control de versiones:** Repositorio Git en GitHub
(<https://github.com/betanS/AfterReload>).

3. Análisis

3.1 Identificación del problema

Las comunidades de Counter-Strike que gestionan sus propios servidores se enfrentan a varios problemas recurrentes:

1. **Fragmentación de herramientas:** La coordinación de jugadores, el control de servidores y el registro de estadísticas se realiza en plataformas distintas y desconectadas.
2. **Gestión manual de lobbies:** La organización de partidas depende de comunicación en canales de texto, con alta probabilidad de errores humanos.
3. **Ausencia de progresión propia:** Los servidores públicos no ofrecen un sistema de puntos vinculado a la identidad del jugador dentro de la comunidad.
4. **Control de acceso rudimentario:** Sin una plataforma propia, es difícil aplicar restricciones (baneos, roles) de forma coherente.

3.2 Requisitos funcionales

RF-01 — Autenticación mediante Steam

El sistema debe permitir que los usuarios inicien sesión utilizando su cuenta de Steam. No se requiere registro manual ni contraseña propia.

RF-02 — Gestión de servidores

Los administradores deben poder crear, editar y eliminar servidores. Cada servidor tiene nombre, IP, puerto, tipo (público o matchmaking), estado (online/offline) y capacidad máxima de jugadores.

RF-03 — Vista de servidores para usuarios

Los usuarios autenticados deben ver la lista de servidores disponibles con el número de jugadores

actual y su estado.

RF-04 — Lobbies para servidores matchmaking

Cada servidor de tipo matchmaking tiene un lobby asociado, restringido a usuarios con rol admin o betatester. Los usuarios pueden unirse seleccionando equipo (CT o T) y marcar su estado de preparación. El lobby se bloquea cuando hay una partida activa.

RF-05 — Lobbies para servidores públicos

Los servidores públicos tienen lobbies de capacidad libre donde los jugadores pueden registrar su presencia. No hay selección de equipo ni inicio automático de partida.

RF-06 — Inicio automático de partida

Cuando al menos la mitad de los jugadores en un lobby matchmaking estén listos y los equipos estén equilibrados, el sistema debe iniciar la partida en el servidor a través de RCON y Get5.

RF-07 — Procesamiento de resultados

Al finalizar una partida (competitiva o pública), el sistema debe registrar el resultado y aplicar puntos y créditos a los jugadores participantes según si ganaron o perdieron.

RF-08 — Procesamiento manual de logs

El sistema debe ofrecer un comando Artisan (`server:process-wins`) que permita forzar el procesamiento de los logs de servidores públicos sin esperar al ciclo automático.

RF-09 — Ranking público

El sistema debe mostrar una tabla con los 50 jugadores con mayor puntuación, accesible sin autenticación.

RF-10 — Panel de administración

Los administradores deben disponer de un panel centralizado para gestionar usuarios (cambiar rol, banear, modificar puntos y créditos) y servidores, con KPIs del sistema en tiempo real.

RF-11 — Internacionalización

La interfaz debe estar disponible en español e inglés, con posibilidad de cambiar el idioma desde la barra de navegación.

RF-12 — Gestión de usuarios baneados

Los usuarios baneados deben ser redirigidos a una página informativa y no deben poder acceder a las funcionalidades del sistema.

RF-13 — Formulario de contacto

El sistema debe ofrecer una página de contacto pública donde cualquier visitante pueda enviar un mensaje al administrador mediante email. El formulario incluye validación AJAX en el cliente.

RF-14 — Eliminación de cuenta

Un usuario autenticado debe poder eliminar permanentemente su cuenta desde el perfil, previa confirmación mediante modal.

RF-15 — Bloqueo de lobby durante partida

Cuando un lobby matchmaking tiene una partida activa, debe quedar bloqueado: no se permite unirse, cambiar equipo ni modificar el estado de preparación.

RF-16 — Filtro de presencia para puntos en servidores públicos

Los puntos derivados de resultados en servidores públicos solo se otorgan a jugadores registrados en el lobby de ese servidor en el momento del procesamiento.

3.3 Requisitos no funcionales

RNF-01 — Rendimiento: La vista de servidores se actualiza cada 5 segundos sin recargar la página. La vista de lobby hace polling cada 3 segundos para reflejar cambios en tiempo real.

RNF-02 — Seguridad: La autenticación se delega a Steam mediante OAuth. Las rutas de administración están protegidas por middleware. Los comandos RCON se envían desde el servidor, nunca desde el cliente. Los tokens de webhook se verifican con comparación segura (`hash_equals`).

RNF-03 — Consistencia de datos: El sistema evita procesar el mismo evento de partida más de una vez mediante hashes SHA-256 como identificadores únicos.

RNF-04 — Mantenibilidad: El código sigue el patrón MVC de Laravel. La lógica compleja se encapsula en servicios independientes.

RNF-05 — Escalabilidad: La base de datos puede configurarse como SQLite (desarrollo) o MySQL/MariaDB (producción).

RNF-06 — Despliegue: El sistema puede instalarse en un servidor Linux estándar con Nginx, PHP-FPM y Composer, sin dependencias propietarias.

3.4 Actores del sistema

Actor	Descripción
Visitante	Puede ver la página de bienvenida, el ranking y el formulario de contacto. No puede acceder a servidores ni lobbies.
Usuario autenticado	Accede a servidores, lobbies públicos, ranking y su perfil.
Beta tester	Usuario con acceso adicional a lobbies de tipo matchmaking.
Administrador	Acceso completo, incluyendo panel de administración, tienda e inventario.
Sistema Get5	Actor externo que envía eventos de partido al webhook de la API.
Servidor CS:GO	Actor externo que genera logs de partidas públicas.

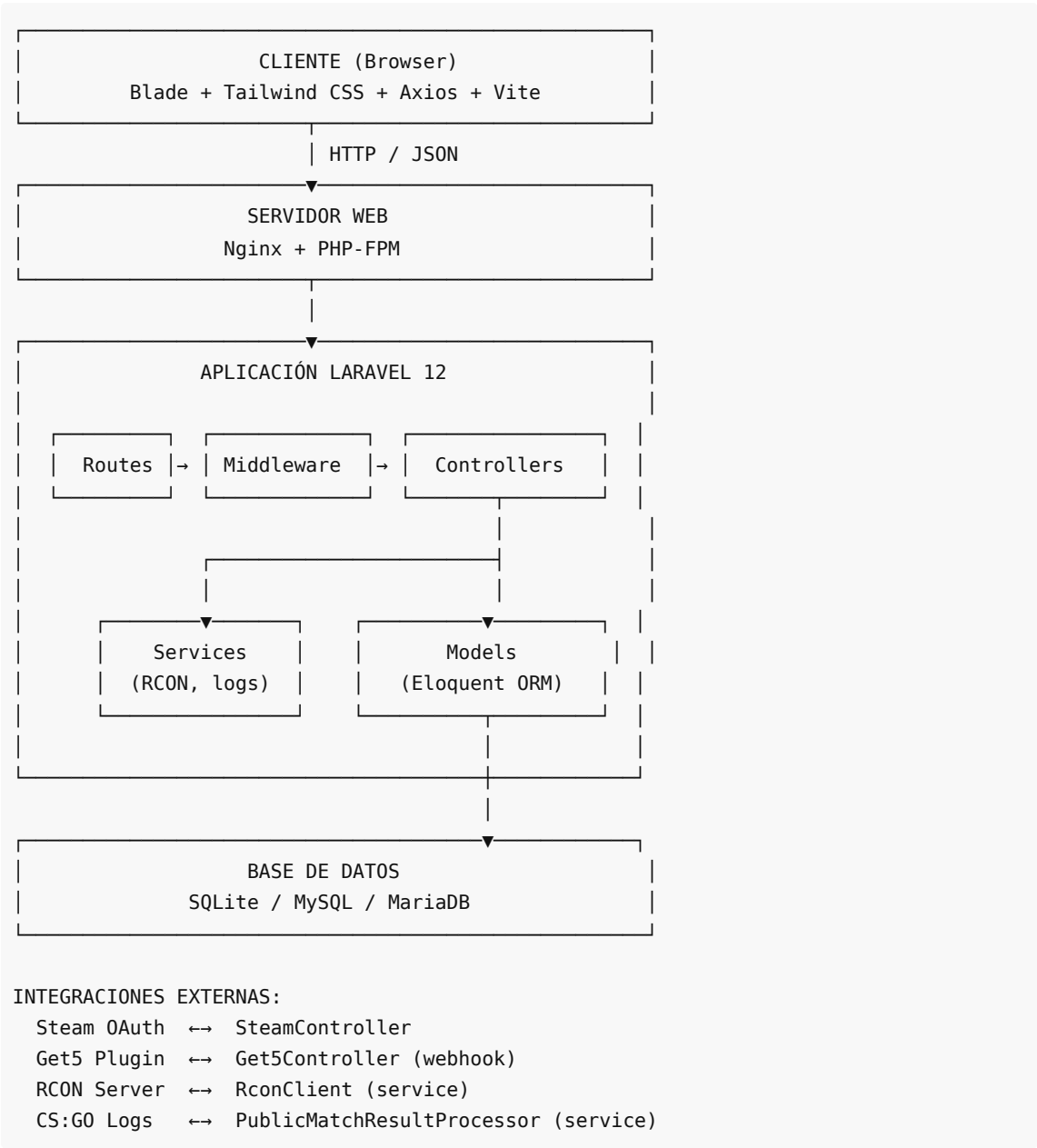
3.5 Restricciones técnicas detectadas

- Los servidores de CS:GO Legacy requieren librerías de 32 bits (`lib32gcc-s1` , `lib32stdc++6`), que deben instalarse en el sistema operativo independientemente de la aplicación web.
- La integración con Get5 requiere que el servidor de juego tenga el plugin instalado y configurado con la URL del webhook.
- El estado de presencia en lobbies tiene un TTL de 20 segundos. Si un usuario cierra el navegador sin salir explícitamente, el sistema lo expulsa automáticamente en el siguiente ciclo de poda.
- La base de datos del servidor de juego (para sincronización opcional de niveles con SourceMod) es una conexión separada de la base de datos principal.

4. Diseño

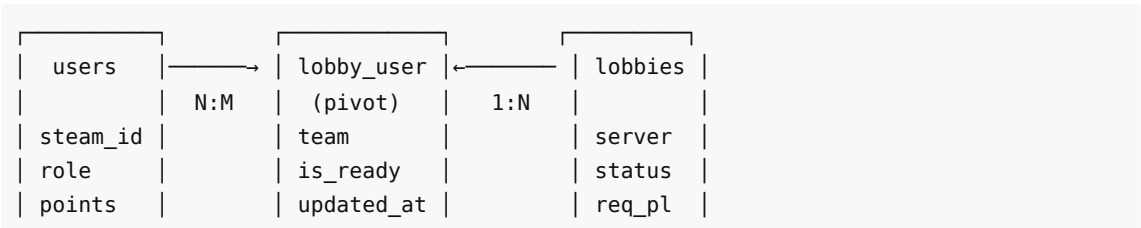
4.1 Arquitectura general

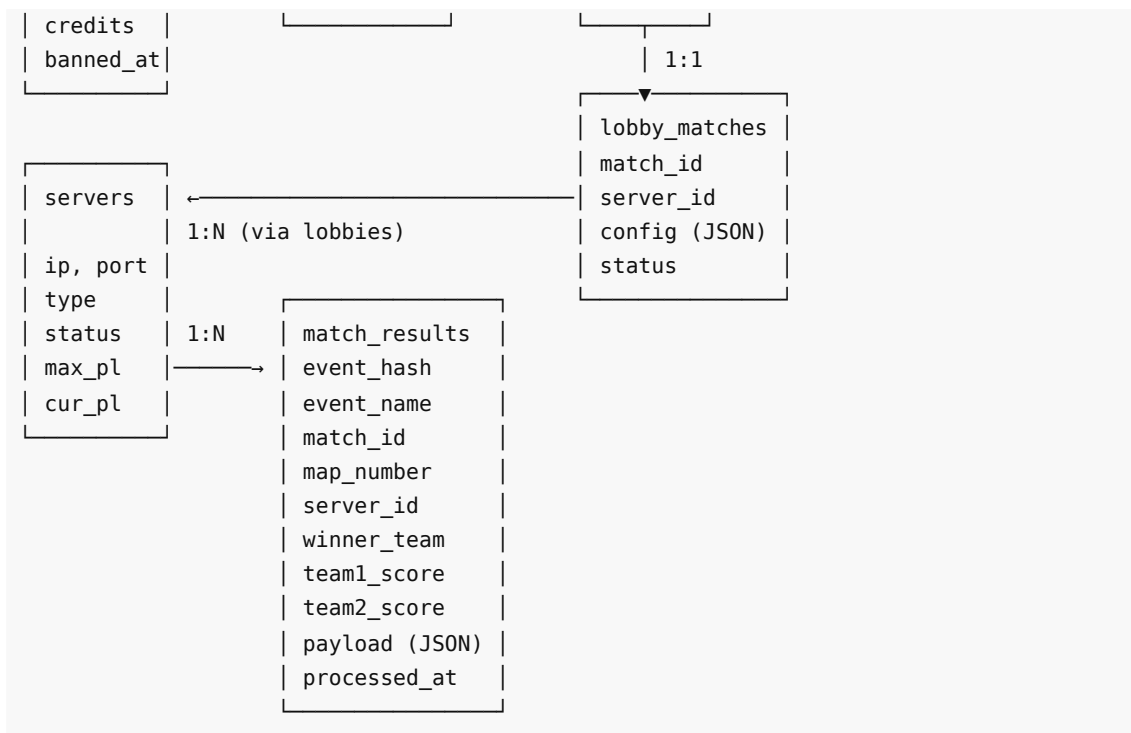
El sistema sigue la arquitectura MVC estándar de Laravel, con capas adicionales de servicios, eventos y middleware para la lógica de negocio compleja:



4.2 Modelo de datos

Diagrama entidad-relación





Descripción de tablas

users — Jugadores del sistema.

- `steam_id` : Identificador único de Steam (cadena de 17 dígitos).
- `steam_nickname` / `steam_real_name` / `avatar` : Datos importados desde Steam.
- `role` : Enumerado `user` | `admin` | `betatester` | `store` .
- `points` / `credits` : Progresión del jugador (enteros, mínimo 0).
- `banned_at` : Si no es null, el usuario está baneado.

servers — Servidores de Counter-Strike.

- `type` : `public` (procesado por logs) o `mm` (matchmaking con Get5).
- `status` : `online` | `offline` (administrado manualmente o por LGSM).
- `max_players` : Capacidad máxima del lobby asociado.
- `current_players` : Contador actualizado automáticamente en cada join/leave.
- `rcon_password` : Contraseña para comandos RCON (nullable).

lobbies — Salas de espera asociadas a servidores.

- `required_players` : Número de jugadores necesarios para iniciar la partida ($\leq$ `max_players` , máximo 10).
- `status` : `waiting` | `live` | `completed` .
- `started_at` : Momento en que comenzó la partida.

lobby_user (pivot) — Relación N:M entre usuarios y lobbies.

- `team` : `ct` | `t` | null.
- `is_ready` : Booleano.
- `updated_at` : Timestamp de presencia, usado para el TTL de 20 segundos.
- Índice único [`lobby_id`, `user_id`] para evitar duplicados.

lobby_matches — Configuración de matches activos.

- `match_id` : Identificador único del match en Get5 (formato `ar-{lobby_id}-{timestamp}`).
- `config` : JSON completo con la configuración de Get5 (equipos, mapa, contraseña de partida, CVARs).
- `server_id` : Servidor asociado al match.
- `status` : `pending` | `live` | `completed` .

match_results — Historial de resultados de partidas.

- `event_hash` : SHA-256 del payload del evento (evita duplicados).
- `event_name` : Tipo de evento (`series_result` , `series_end` , `map_result` , `map_end` , `public_log_result` , etc.).
- `match_id` / `map_number` : Identificadores de la partida y el mapa.
- `server_id` : Servidor de origen del evento.
- `winner_team` / `team1_score` / `team2_score` : Resultado.
- `payload` : JSON completo del evento para auditoría.
- `processed_at` : Timestamp del procesamiento.

4.3 Diseño de rutas

Rutas web (`routes/web.php`)

GET /	→ Página de bienvenida con contador de usuarios (pública)
GET /lang/{locale}	→ Cambio de idioma
GET /login	→ Redirección a Steam
GET /login/steam	→ Inicio OAuth Steam
GET /login/steam/callback	→ Callback OAuth Steam
GET /ranking	→ Ranking de jugadores (pública)
GET /contact	→ Página de contacto (pública)
POST /contact	→ Enviar mensaje de contacto por email (pública)
[Middleware: auth]	
GET /banned	→ Página de usuario baneado
GET /profile	→ Perfil del usuario autenticado
GET /inventory	→ Inventario (admin: completo; otros: "Próximamente")
GET /store	→ Tienda (admin: completo; otros: "Próximamente")
GET /servers	→ Lista de servidores
GET /servers/data	→ JSON con estado de servidores (polling cada 5s)
GET /lobby/{server}	→ Vista de lobby
GET /lobby/{server}/status	→ JSON con estado del lobby (polling cada 3s)
POST /lobby/{server}/leave	→ Abandonar lobby
POST /lobby/{server}/team	→ Seleccionar equipo / unirse a lobby público
POST /lobby/{server}/ready	→ Alternar estado de preparación
DELETE /account	→ Eliminar cuenta permanentemente
POST /logout	→ Cerrar sesión
[Middleware: auth, admin]	
GET /admin	→ Panel de administración
POST /admin/servers	→ Crear servidor
PUT /admin/servers/{server}	→ Editar servidor
DELETE /admin/servers/{server}	→ Eliminar servidor
POST /admin/servers/clear-lobbies	→ Limpiar lobbies (por tipo: mm o public)

POST /admin/users/{user}/role	→ Cambiar rol
POST /admin/users/{user}/ban	→ Banear usuario
POST /admin/users/{user}/unban	→ Desbanear usuario
POST /admin/users/{user}/stats	→ Editar puntos y créditos

Rutas API (routes/api.php)

POST /api/get5/events	→ Webhook de eventos Get5
GET /api/get5/match/{lobby}	→ Configuración de match para Get5

4.4 Diseño de la interfaz

La interfaz sigue un diseño oscuro cohesionado con la estética del videojuego, usando Tailwind CSS 4.0 directamente en las plantillas Blade.

Layout principal

TOPBAR: Logo Selector de idioma Avatar + nombre	
SIDEBAR	CONTENIDO PRINCIPAL
Servidores	
Ranking	
Tienda	
Inventario	
Perfil	
Contacto	
[Admin]	

Descripción de vistas

welcome.blade.php — Página de inicio pública. Muestra el nombre del proyecto, una descripción breve, el número total de jugadores registrados en la plataforma y el botón de inicio de sesión mediante Steam. Si el usuario ya está autenticado, el botón redirige a la lista de servidores.

home.blade.php / servidores — Lista de todos los servidores configurados. Muestra por cada servidor: nombre, tipo (público o matchmaking), estado (online/offline), jugadores actuales y capacidad máxima. Los datos se actualizan automáticamente cada 5 segundos mediante polling AJAX sin recargar la página. Los servidores online son clicables y llevan al lobby correspondiente.

lobby.blade.php — Vista principal de lobby. Se divide en dos columnas (CT y T) con los avatares y nombres de los jugadores de cada equipo, su estado de preparación (listo/no listo) y un indicador de cuántos jugadores faltan para iniciar. Incluye botones para unirse a un equipo, marcar/desmarcar el estado de preparación y abandonar el lobby. En servidores matchmaking, cuando la partida está activa, el lobby muestra el estado bloqueado y la contraseña de conexión al servidor. En servidores públicos, el botón de unirse no requiere selección de equipo. El estado se actualiza cada 3 segundos mediante polling.

Lobby: Servidor X [5/10 jugadores listos]	
COUNTER-TERRORISTS	TERRORISTS
[Avatar] Jugador1 ✓	[Avatar] Jugador3 ✓
[Avatar] Jugador2 x	[Avatar] Jugador4 x
[Unirse CT]	[Unirse T]
[Marcar como listo / No listo] [Abandonar lobby]	
IP: 0.0.0.0:27015 Contraseña: **** (live)	

ranking.blade.php — Tabla pública con los 50 jugadores con mayor puntuación. Muestra posición, avatar, nombre de Steam, puntos y créditos. Accesible sin autenticación.

contact.blade.php — Formulario de contacto público con campos de nombre, email, asunto y mensaje. Incluye validación AJAX en tiempo real antes de enviar. Al enviar correctamente muestra un mensaje de éxito en la propia página sin recargar. No requiere autenticación.

profile.blade.php — Perfil del usuario autenticado. Muestra el avatar de Steam, nombre, puntos y créditos acumulados. Incluye un botón de eliminación de cuenta que abre un modal de confirmación antes de ejecutar la acción. La eliminación es permanente e irreversible.

inventory.blade.php — Módulo de inventario. Los usuarios con rol `admin` ven el contenido completo del inventario. El resto de usuarios ven una pantalla de "Próximamente" indicando que la funcionalidad está en desarrollo.

store.blade.php — Módulo de tienda. Idéntico en comportamiento de acceso al inventario: solo admins acceden al contenido completo; el resto ve "Próximamente".

admin/index.blade.php — Panel de administración centralizado. Muestra KPIs del sistema en tarjetas (total de usuarios, lobbies activos, servidores configurados, partidas jugadas, servidores online). Incluye dos tablas principales: gestión de servidores (con edición inline mediante formularios modales, creación y eliminación) y gestión de usuarios (con cambio de rol, baneo/desbaneo y modificación directa de puntos y créditos). También incluye el botón de limpieza de lobbies diferenciado por tipo (matchmaking o público).

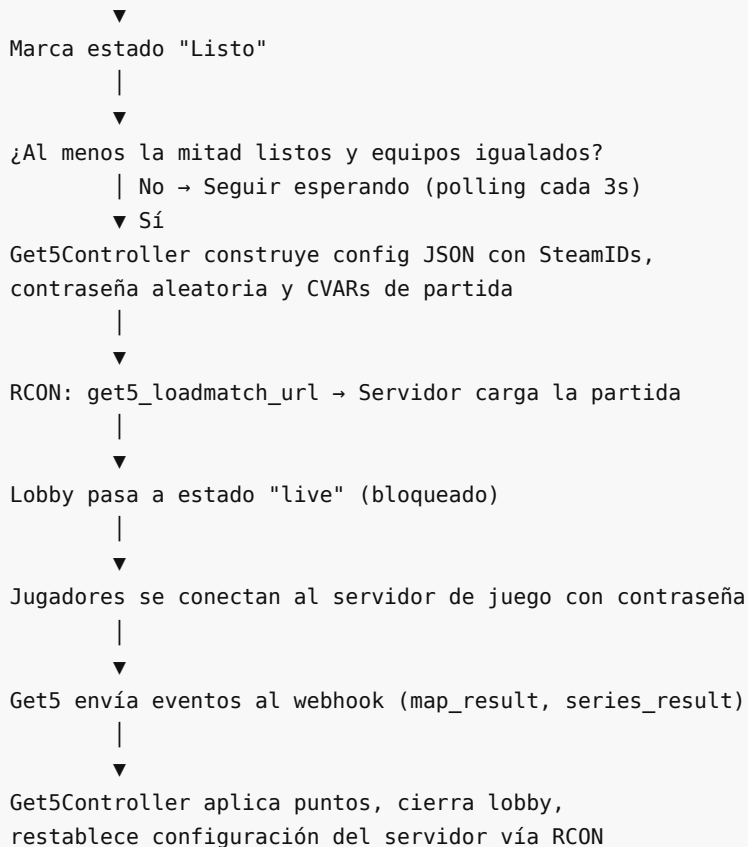
banned.blade.php — Página informativa que se muestra a los usuarios baneados. Indica que la cuenta ha sido suspendida y ofrece únicamente la opción de cerrar sesión.

4.5 Flujo de matchmaking completo

```

Usuario entra al lobby (debe ser admin o betatester)
|
▼
¿Ya existe un lobby activo para ese servidor?
| No → Se crea el lobby automáticamente
▼ Sí
Selecciona equipo (CT / T)
|

```



5. Implementación

5.1 Estructura de directorios

```
afterreload/
├─ app/
│   ├── Console/
│   │   └─ Commands/
│   │       └─ ProcessServerWins.php      # Comando artisan manual
│   ├── Events/
│   │   └─ LobbyUpdated.php              # Evento broadcastable
│   ├── Http/
│   │   └─ Controllers/
│   │       ├── Admin/
│   │       │   └─ AdminController.php
│   │       ├── Api/
│   │       │   └─ Get5Controller.php
│   │       ├── Auth/
│   │       │   └─ SteamController.php
│   │       ├── Lobby/
│   │       │   └─ LobbyController.php
│   │       ├── AccountController.php
│   │       ├── ContactController.php
│   │       ├── LanguageController.php
│   │       └─ RankingController.php
```

```
| | | └─ ServerController.php
| | |   └─ Middleware/
| | |     └─ EnsureNotBanned.php
| | |     └─ SetLocale.php
| | └─ Mail/
| |   └─ ContactReport.php
| └─ Models/
|   └─ Lobby.php
|   └─ LobbyMatch.php
|   └─ MatchResult.php
|   └─ Server.php
|   └─ User.php
| └─ Services/
|   └─ LocalCsgoserverRuntimeService.php
|   └─ PublicMatchResultProcessor.php
|   └─ RconClient.php
└─ database/
  └─ migrations/                                # 10 migraciones versionadas
└─ lang/
  └─ en.json
  └─ es.json
└─ resources/
  └─ views/
    └─ layouts/app.blade.php
    └─ partials/
      └─ navbar.blade.php
      └─ sidebar.blade.php
      └─ topbar.blade.php
    └─ emails/
      └─ contact-report.blade.php
    └─ admin/index.blade.php
    └─ banned.blade.php
    └─ contact.blade.php
    └─ inventory.blade.php
    └─ lobby.blade.php
    └─ profile.blade.php
    └─ ranking.blade.php
    └─ store.blade.php
    └─ welcome.blade.php
└─ routes/
  └─ api.php
  └─ web.php
└─ tests/
  └─ Feature/
    └─ LobbyTest.php
  └─ Unit/
    └─ PublicMatchResultProcessorTest.php
└─ composer.json
└─ package.json
└─ vite.config.js
```

5.2 Módulo de autenticación con Steam

La autenticación implementa el flujo OAuth 2.0 a través del paquete `socialiteproviders/steam`. El proceso es el siguiente:

1. El usuario hace clic en "Iniciar sesión con Steam".
2. `SteamController::redirectToSteam()` redirige al servidor de autenticación de Steam.
3. Steam autentica al usuario y redirige de vuelta al callback de la aplicación.
4. `SteamController::handleSteamCallback()` recibe el token y crea o actualiza el registro del usuario en la base de datos con los datos del perfil de Steam (nickname, avatar, SteamID64).
5. La sesión se inicia y el usuario es redirigido a la vista de servidores.

La verificación de administrador se realiza de dos formas complementarias: mediante el campo `role` en la base de datos o mediante la lista `ADMIN_STEAM_IDS` en las variables de entorno, actuando esta última como fallback de recuperación de acceso.

El middleware `EnsureNotBanned` se aplica a todas las rutas autenticadas. Si el campo `banned_at` del usuario no es nulo, redirige a `/banned` independientemente de la ruta solicitada.

5.3 Módulo de lobbies

El `LobbyController` es el componente más complejo del sistema. Sus responsabilidades incluyen:

Poda de usuarios inactivos (TTL): Los registros en `lobby_user` tienen un timestamp de actualización. Si no se actualiza en 20 segundos, el registro se elimina. Cada petición al endpoint de status renueva el timestamp. Si el cliente cierra el navegador, el registro expira en el siguiente ciclo de poda.

Verificación de equipos: Antes de iniciar un match, el sistema verifica que ambos equipos tengan el mismo número de jugadores y que al menos la mitad estén marcados como listos.

Bloqueo durante partida (`locked`): Cuando un lobby matchmaking tiene una `LobbyMatch` activa y `started_at` no es nulo, el payload devuelve `locked: true`. Las acciones de unirse, cambiar equipo y alternar el estado de preparación quedan deshabilitadas en el cliente.

Lobbies públicos (`is_unlimited`): Los servidores de tipo `public` no restringen equipos ni requieren equilibrio para iniciar partida. Al unirse, el equipo se asigna automáticamente como `ct` internamente, pero la vista no muestra división por equipos.

Control de acceso por rol: Los lobbies de tipo `mm` verifican que el usuario sea `admin` o `betatester` mediante `ensureBetaAccess()`. El resto recibe un error 403.

Broadcasting en tiempo real: Tras cada operación que modifica el lobby, se emite el evento `LobbyUpdated` en el canal `lobby.{server_id}` (nombre de evento: `LobbyUpdated`). El evento implementa `ShouldBroadcastNow`, lo que significa que se emite de forma inmediata sin pasar por la cola. Si el broadcasting está configurado (Pusher/Soketi), el cliente recibe el estado actualizado sin esperar el ciclo de polling.

Contador de jugadores: Tras cada join/leave, se actualiza el campo `current_players` del servidor con el máximo de jugadores entre todos sus lobbies activos.

5.4 Módulo de integración con Get5

El `Get5Controller` gestiona dos puntos de entrada:

GET /api/get5/match/{lobby} : Construye y devuelve la configuración JSON que Get5 necesita para cargar la partida. El JSON incluye:

- `matchid` : identificador único con formato `ar-{lobby_id}-{timestamp}` .
- `players_per_team` : tamaño de cada equipo.
- `min_players_to_ready` : total de jugadores en lobby (todos deben confirmar).
- `maplist` : mapa seleccionado (configurable vía `GET5_DEFAULT_MAP`).
- `password` : contraseña aleatoria de 8 caracteres hexadecimales generada en el momento.
- `cvars` : `get5_auto_ready 1` , `get5_move_to_spec_on_ready 1` , `get5_skip_knife_round 1` .
- `team1 / team2` : mapas `{SteamID → nickname}` de cada equipo.

El endpoint acepta el token de autenticación en la cabecera `Authorization: Bearer TOKEN` o en `X-Get5-Token TOKEN` .

POST /api/get5/events : Recibe eventos del match. Los eventos `series_result` y `series_end` desencadenan la aplicación de puntos (+5 puntos y +2 créditos a ganadores; -4 puntos a perdedores) y el cierre del lobby. Tras el cierre, el sistema envía comandos RCON al servidor para restablecer los plugins de personalización que Get5 deshabilita durante la partida.

La deduplicación de eventos se realiza mediante el hash SHA-256 del payload completo. Si el hash ya existe en `match_results` , el evento se descarta sin procesarlo.

Opcionalmente, si está configurada la conexión `DB_SERVER_*` , los puntos se sincronizan también en la tabla `lvl_base` de la base de datos del plugin SourceMod `lvl_ranks` , manteniendo la progresión coherente entre la plataforma web y el servidor de juego.

5.5 Servicio RCON

`RconClient` implementa el protocolo RCON de Valve directamente en PHP, sin dependencias externas. El protocolo funciona sobre TCP con un formato de paquete binario:

```
[longitud: 4 bytes][id: 4 bytes][tipo: 4 bytes][datos: variable][terminador: 2 bytes]
```

El cliente gestiona apertura/cierre del socket, autenticación con contraseña, envío de comandos y lectura de respuestas. El timeout por defecto es de 1.5 segundos; si el servidor no responde, `send()` devuelve `null` sin lanzar excepción.

Momento	Comando enviado
Inicio de match	<code>get5_server_id "{server_id}" + get5_loadmatch_url "URL" "Authorization" "Bearer TOKEN"</code>
Fin de match	Restauración de plugins de skins (<code>sm_ws</code> , <code>sm_gloves</code> , <code>sm_agents</code> , <code>sm_weapons_table_prefix</code>)

5.6 Servicio de procesamiento de logs públicos

`PublicMatchResultProcessor` analiza los ficheros de log del servidor de CS:GO para detectar el final de partidas en servidores públicos. El proceso es:

1. Se lee cada fichero de log en el directorio configurado.
2. Se busca la línea que indica fin de partida mediante expresión regular.

3. Se determina el umbral de victoria: 9 rondas (Wingman, puerto 27016 o `max_players ≤ 4`) o 16 rondas (5v5 estándar).
4. Se extraen los SteamIDs de los jugadores de cada equipo desde las líneas de log anteriores.
5. **Filtro de presencia:** solo se puntúan los jugadores cuyos SteamIDs estén registrados en el lobby activo de ese servidor en ese momento.
6. Se aplican puntos (+5 y +2 créditos a ganadores; −4 puntos a perdedores, mínimo 0).
7. Se registra el resultado en `match_results` con hash de deduplicación.
8. Un lock de caché evita que dos procesos simultáneos procesen el mismo log.

El servicio se ejecuta automáticamente con un intervalo mínimo de 15 segundos entre ejecuciones, disparado por cualquier acceso al lobby de un servidor público.

5.7 Comando Artisan de procesamiento manual

El comando `php artisan server:process-wins` permite ejecutar el `PublicMatchResultProcessor` de forma manual, sin esperar al ciclo automático. Acepta el flag `--force` para ignorar el throttle de 15 segundos y procesar inmediatamente aunque el servicio haya corrido recientemente.

```
# Ejecución normal (respeta el intervalo de 15 segundos)
php artisan server:process-wins

# Forzar procesamiento inmediato
php artisan server:process-wins --force
```

Este comando es útil para administradores que necesiten procesar resultados de partidas que el sistema no haya detectado en el ciclo automático, o para verificar el funcionamiento del procesador en entornos de prueba.

5.8 Módulo de contacto

`ContactController` gestiona el formulario de contacto público disponible en `/contact`. La validación se realiza en dos niveles:

- **Validación AJAX en cliente:** el JavaScript verifica los campos antes de enviar, mostrando errores inline.
- **Validación en servidor:** `ContactController::send()` aplica las mismas reglas mediante el sistema de validación de Laravel.

Al superar la validación, se construye un `ContactReport` (clase `Mailable`) y se envía a la dirección del administrador mediante Laravel Mail. El envío está envuelto en un bloque `try/catch` que descarta silenciosamente cualquier fallo SMTP, garantizando que el usuario siempre recibe un mensaje de éxito.

5.9 Módulo de eliminación de cuenta

`AccountController::destroy()` gestiona la eliminación permanente de cuentas. El flujo es: el usuario confirma en un modal → se envía `DELETE /account` → el controlador cierra la sesión, elimina el registro de la tabla `users` y redirige a la página de bienvenida.

5.10 Panel de administración

El panel de administración (`AdminController`) ofrece:

- **KPIs:** Total de usuarios, lobbies activos, servidores configurados, partidas jugadas y servidores online en tiempo real.
- **Gestión de servidores:** CRUD completo con edición inline mediante formularios modales. La limpieza de lobbies diferencia entre tipo `mm` y `public`, actualizando el estado de todos los lobbies y reseteando `current_players` en el servidor.
- **Gestión de usuarios:** Tabla paginada (50 por página) con acciones de cambio de rol, baneo/desbaneo y edición directa de puntos y créditos.

El acceso está protegido por middleware que verifica el rol `admin` o la presencia del SteamID en `ADMIN_STEAM_IDS`.

5.11 Sistema de internacionalización

Todas las cadenas visibles están externalizadas en `lang/en.json` y `lang/es.json`. El middleware `SetLocale` aplica el locale de Laravel en cada petición a partir de la sesión del usuario. El selector de idioma en la barra de navegación actualiza la sesión sin recargar la página.

6. Validación

6.1 Validación de flujos funcionales

Flujo de autenticación

- El usuario accede sin autenticar, ve la página de bienvenida con el contador de usuarios.
- Al hacer clic en "Iniciar sesión", es redirigido a Steam.
- Tras autenticarse, vuelve como usuario registrado con avatar y nombre de Steam.
- El cierre de sesión elimina la sesión y redirige a la página de bienvenida.
- Un usuario baneado es redirigido a `/banned` en cualquier ruta protegida.

Flujo de lobby matchmaking

- Un usuario con rol `betatester` o `admin` accede al lobby.
- Puede seleccionar equipo (CT o T) y cambiar su estado de preparación.
- Si cierra el navegador sin salir, el sistema lo elimina del lobby en el siguiente ciclo (20s).
- Cuando se cumplen las condiciones, el sistema inicia el match, el lobby se bloquea y se muestra la contraseña de conexión.
- Al finalizar la partida, Get5 notifica al webhook, se aplican puntos y el lobby vuelve a `waiting`.

Flujo de lobby público

- Cualquier usuario autenticado puede unirse al lobby pulsando el botón de join.
- Al finalizar la partida en el servidor, el sistema procesa el log y puntúa solo a los jugadores presentes en el lobby.

Flujo de contacto

- El formulario valida los campos en tiempo real (AJAX).
- Al enviar correctamente, el administrador recibe el email y el usuario ve el mensaje de éxito.
- Si el SMTP falla, el usuario igualmente ve el mensaje de éxito (fallo silencioso).

Flujo de administración

- Un administrador puede crear, editar y eliminar servidores.
- Puede cambiar el rol de un usuario, banearlo/desbanearlo y ajustar puntos y créditos.

- La limpieza de lobbies vacía correctamente los registros y resetea los contadores.

6.2 Validación de la integridad de datos

- Los `steam_id` son únicos en `users` . Si un usuario existente vuelve a autenticarse, se actualiza su registro en lugar de crear uno nuevo.
- Los registros de `lobby_user` tienen restricción de unicidad sobre `[lobby_id, user_id]` .
- El campo `event_hash` en `match_results` es único, garantizando que ningún resultado se procese más de una vez.
- El campo `current_players` se mantiene siempre sincronizado con el número real de jugadores en el lobby activo.

6.3 Validación del entorno de instalación

La instalación del sistema en un servidor limpio fue validada siguiendo el procedimiento documentado en el Manual Técnico:

1. Instalación de dependencias del sistema.
2. Clonado del repositorio desde `https://github.com/betanS/AfterReload` .
3. `composer install` y `npm install` para dependencias.
4. Configuración del fichero `.env` .
5. `php artisan migrate --force` para crear el esquema.
6. `npm run build` para compilar el frontend.
7. Configuración de Nginx y PHP-FPM.
8. Verificación de accesibilidad y funcionalidad.

7. Pruebas

7.1 Estrategia de pruebas

El proyecto combina pruebas automatizadas con PHPUnit para los componentes de lógica pura y de integración, y pruebas manuales para los flujos que requieren integraciones externas (Steam, Get5, RCON).

7.2 Pruebas unitarias — Procesador de logs

Las pruebas unitarias se ubican en `tests/Unit/PublicMatchResultProcessorTest.php` y cubren el servicio de procesamiento de logs de servidores públicos.

Caso	Método	Estado
Victoria CT 5v5 (Bomb Defused)	<code>test_extract_finished_match_supports_bomb_defused_results_for_public_5v5</code>	Pasa
No procesar ronda no terminal	<code>test_extract_finished_match_ignores_non_terminal_round_wins</code>	Pasa
Victoria CT Wingman (9 rondas)	<code>test_extract_finished_match_supports_wingman_ct_win_at_nine_rounds</code>	Pasa

Victoria T Wingman (9 rondas)	test_extract_finished_match_supports_wingman_t_win_at_nine_rounds	Pasa
Descarte no terminal Wingman	test_extract_finished_match_ignores_wingman_non_terminal_loss_lines	Pasa
Resolución umbral por puerto	test_resolve_winning_score_uses_wingman_threshold_for_27016_logs	Pasa

7.3 Pruebas de integración — Lobby

Las pruebas de integración se ubican en `tests/Feature/LobbyTest.php` y cubren el flujo de acceso y join al lobby usando una base de datos en memoria (`RefreshDatabase`).

Caso	Método	Estado
Vista de lobby contiene todas las variables requeridas	test_lobby_view_has_all_required_variables	Pasa
Acceder al lobby no une automáticamente al usuario	test_opening_lobby_does_not_auto_join_user	Pasa
Elegir equipo registra al usuario en el lobby	test_joining_team_confirms_lobby_attendance	Pasa

Se ejecutan con:

```
php artisan test
```

7.4 Pruebas manuales realizadas

Funcionalidad	Acción	Resultado esperado	Verificado
Autenticación	Login con cuenta Steam real	Usuario creado/actualizado en BD	Sí
Autenticación	Cierre de sesión	Sesión destruida, redirección a inicio	Sí
Lobby matchmaking	Entrar a lobby sin rol	Error 403	Sí
Lobby matchmaking	Seleccionar equipo CT	Usuario asignado a CT	Sí
Lobby matchmaking	Marcar como listo	Estado de preparación actualizado	Sí
Lobby matchmaking	Cerrar navegador	Usuario eliminado tras 20s	Sí

Lobby matchmaking	Partida iniciada	Lobby bloqueado, contraseña visible	Sí
Lobby público	Unirse al servidor	Usuario aparece en lobby	Sí
Lobby público	Abandonar lobby	Jugador eliminado, contador actualizado	Sí
Procesamiento logs	<code>server:process-wins --force</code>	Resultado procesado y puntos aplicados	Sí
Contacto	Formulario con datos válidos	Email recibido por el administrador	Sí
Contacto	Email inválido	Error inline sin recargar	Sí
Perfil	Eliminar cuenta	Cuenta eliminada, redirección a inicio	Sí
Inventario	Acceso usuario normal	Vista "Próximamente"	Sí
Admin	Crear/editar/eliminar servidor	Cambios reflejados en lista	Sí
Admin	Banear usuario	Usuario redirigido a /banned	Sí
Admin	Limpiar lobbies	Lobbies vaciados por tipo	Sí
Ranking	Ver ranking	Top 50 por puntos mostrado	Sí
Idioma	Cambiar a inglés/español	Interfaz en el idioma seleccionado	Sí

7.5 Pruebas de integración con Get5 (manual)

La integración con Get5 fue validada mediante el envío manual de payloads JSON al endpoint del webhook:

- Se simuló un evento `series_result` con equipos ficticios.
- Se verificó que el sistema registró el resultado en `match_results`.
- Se verificó que los puntos se aplicaron correctamente a los jugadores.
- Se verificó la protección contra duplicados enviando el mismo payload dos veces.

8. Resultados

8.1 Estado del sistema al finalizar el proyecto

AfterReload es una plataforma web funcional y desplegada que resuelve de forma efectiva los objetivos planteados en la fase de análisis. El sistema está en uso activo por la comunidad para la que fue diseñado.

Funcionalidades implementadas y operativas:

- Autenticación completa mediante Steam OAuth.
- Gestión de servidores con diferenciación de tipos, estados y capacidad configurable.

- Sistema de lobbies con presencia activa, selección de equipo, estado de preparación, bloqueo durante partida y broadcasting de eventos en tiempo real.
- Lobbies públicos con join libre, filtro de presencia para asignación de puntos y conteo automático de jugadores.
- Integración con Get5 para el ciclo completo de partidas competitivas, incluyendo configuración de match con contraseña aleatoria y reset automático del servidor al finalizar.
- Procesamiento automático y manual de resultados en servidores públicos con filtro de presencia.
- Sistema de puntos y créditos para la progresión del jugador.
- Ranking público de los 50 mejores jugadores.
- Formulario de contacto público con envío de email vía SMTP y validación AJAX.
- Eliminación voluntaria de cuenta desde el perfil.
- Panel de administración centralizado con KPIs en tiempo real y limpieza de lobbies por tipo.
- Módulo de inventario y tienda con acceso restringido a administradores.
- Interfaz completamente traducida al español e inglés.
- 9 pruebas automatizadas con PHPUnit (6 unitarias + 3 de integración), todas en estado pasado.

8.2 Métricas del proyecto

Métrica	Valor
Líneas de código PHP (app/)	~1.700 líneas
Controladores	9
Servicios	3
Comandos Artisan personalizados	1
Clases Mail	1
Modelos Eloquent	5
Eventos broadcastables	1
Migraciones	10
Rutas definidas	27
Vistas Blade	15
Claves de traducción	142+
Pruebas automatizadas	9 (6 unit + 3 feature)
Tablas en base de datos	11

8.3 Logros destacados

Reducción de dependencias externas: El sistema centraliza la gestión que antes requería múltiples herramientas dispersas.

Consistencia de lobbies: El sistema de presencia con TTL de 20 segundos resolvió el problema crónico de los jugadores fantasma, que anteriormente requería intervención manual.

Deduplicación de eventos: El mecanismo de hash SHA-256 garantiza que ningún resultado de partida se procese más de una vez, aunque el servidor de juego reenvíe el mismo evento.

Integración RCON nativa: La implementación del protocolo RCON en PHP puro elimina dependencias externas para el control remoto de servidores.

Filtro de presencia en servidores públicos: Solo los jugadores registrados en el lobby de la plataforma reciben puntos, vinculando la progresión a la participación activa en el sistema.

Broadcasting en tiempo real: El evento `LobbyUpdated` emitido en el canal `lobby.{serverId}` permite actualizaciones instantáneas vía WebSockets cuando el broadcasting está configurado, como alternativa al polling periódico.

8.4 Limitaciones identificadas

- **Tiempo real mediante polling:** La actualización de servidores y lobbies se realiza mediante polling periódico. El broadcasting via WebSockets está implementado pero requiere infraestructura adicional (Pusher o Sockets) para activarse.
- **Estadísticas avanzadas:** El sistema registra puntos y créditos, pero no estadísticas detalladas por partida (kills, muertes, asistencias).
- **Observabilidad:** No existe un panel de monitorización. Los errores se registran en los logs de Laravel.
- **Cobertura de pruebas:** Las pruebas automatizadas cubren el procesador de logs y el flujo básico del lobby. La lógica de Get5 y RCON dependen de verificación manual.

8.5 Líneas de trabajo futuras

1. **WebSockets como modo principal:** Reemplazar el polling por actualizaciones push mediante Laravel Echo con Pusher o Sockets.
2. **Estadísticas detalladas por partida:** Procesar los logs de Get5 para extraer estadísticas individuales por jugador.
3. **Sistema de notificaciones:** Alertas al usuario cuando su lobby está a punto de comenzar.
4. **Cobertura de pruebas ampliada:** Añadir pruebas de integración para `Get5Controller` y `LobbyController`.
5. **API pública documentada:** Exponer una API REST documentada con OpenAPI/Swagger para integraciones externas.

Anexos

Anexo A — Instrucciones de instalación

Consultar el Manual Técnico (`Manual_Tecnico_AfterReload.pdf`) para las instrucciones detalladas de instalación, configuración y administración del sistema.

Anexo B — Variables de entorno

Las variables mínimas requeridas para un despliegue básico:

```
APP_NAME=AfterReload
APP_URL=https://tu-dominio.com
DB_CONNECTION=sqlite

# Steam OAuth (obligatorio)
```

```
STEAM_CLIENT_SECRET=tu_steam_api_key
STEAM_REDIRECT_URL=https://tu-dominio.com/login/steam/callback

# Administradores (obligatorio)
ADMIN_STEAM_IDS=STEAMID64,STEAMID64

# Integración con Get5 (para matchmaking)
GET5_WEBHOOK_TOKEN=token_secreto_aleatorio
GET5_DEFAULT_MAP=de_mirage

# Control remoto RCON (para matchmaking)
RCON_HOST=127.0.0.1
RCON_PORT=27015
RCON_PASSWORD=tu_rcon_password

# Formulario de contacto – envío de emails
MAIL_MAILER=smtplib
MAIL_HOST=smtplib.gmail.com
MAIL_PORT=587
MAIL_USERNAME=tu_cuenta@gmail.com
MAIL_PASSWORD=tu_app_password_de_google
MAIL_FROM_ADDRESS=tu_cuenta@gmail.com
MAIL_FROM_NAME=AfterReload
```

Anexo C — Comandos de mantenimiento

```
# Ejecutar migraciones
php artisan migrate

# Limpiar caché
php artisan optimize:clear

# Compilar frontend para producción
npm run build

# Ejecutar pruebas
php artisan test

# Procesar logs de servidores públicos manualmente
php artisan server:process-wins
php artisan server:process-wins --force

# Entorno de desarrollo completo (servidor + cola + logs + vite)
composer run dev
```

Anexo D — Repositorio

El código fuente del proyecto está disponible en:

<https://github.com/betanS/AfterReload>

Documentación del proyecto AfterReload — Plataforma de matchmaking para Counter-Strike.
Autor: Alejandro | Mayo 2026